Practicum No.8 **Date:27/09/2026**

Implementation of Unification in Logic Systems

I. Practical Significance

Unification enables AI systems to match logical expressions by finding suitable substitutions for variables. In this practicum, students will implement unification in Python to match predicates and variables, providing a foundation for understanding Prolog, expert systems, and automated reasoning, with industry-relevant applications in Logic Programming Engineering, Knowledge Representation, AI Research, and Rule-Based System Development.

II. Competency and Industry-Specific Practical Skills

This practicum exercise is expected to develop the following skills for the industry-identified competency.

- ***Use*** the principles of artificial intelligence to solve complex engineering problems.
  (As stated in the Course Structure)
  a. ***Apply*** logical reasoning and knowledge representation methods for building rule-based systems and intelligent agents. *(Cognitive Domain)*
  b. ***Use*** probabilistic reasoning models, such as Bayesian networks, for decision-making under uncertainty. *(Cognitive Domain)*
  c. ***Interpret*** AI model outcomes and ensure explainability in decision-making systems. *(Cognitive Domain)*
  d. ***Demonstrate*** accountability and professionalism while working in interdisciplinary AI development teams. *(Affective Domain)*

III. Relevant CO(s) *(As stated in the Course Structure)*

- **CO3. *Use*** suitable knowledge representation techniques for AI-based applications.

IV. Practical Outcome (PrO) *(As stated in the Course Structure)*

- ***Develop*** the Unification Algorithm for ***the given*** logical expressions and demonstrate its application.

V. Related ADO(s) *(As stated in the Course Structure)*

a) ***Follow*** ethical practices in AI model development and deployment.
b) ***Follow*** adaptability to emerging AI techniques and technologies.
c) ***Follow*** a structured approach to AI driven decision making.
d) ***Follow*** a commitment to continuous learning.

VI. Minimum Underpinning Theory *(Include relevant content & images for this practicum.)*

Unification is the process of making two logical expressions identical by finding suitable substitutions for their variables. It is a key mechanism for inference in First-Order Logic (FOL) and is fundamental to automated reasoning, theorem proving, and logic programming such as Prolog. A unifier is a set of substitutions that, when applied to two expressions, makes them identical.

Key Concepts:

Variable Matching: Replacing variables with constants or other variables to make terms identical.
Substitution: A mapping of variables to constants, variables, or other terms.
Occurs Check: Ensuring that a variable does not occur within the term to which it is being substituted, preventing invalid self-referential expressions.

VII. Practicum Set-up/Circuit Diagram/Algorithm/Flowchart *(Include probable photos of experimental set-up.)*

Algorithm:

1. Take two logical expressions as input.
2. Compare their corresponding terms.
3. If both terms are identical, continue.
4. If a term is a variable, substitute it with the corresponding term.
5. If both are constants, check whether they are equal; otherwise, return **FAIL**.
6. For compound terms, compare their function names and arguments recursively.

7. Apply the substitutions to all remaining terms.
8. Return the final substitution set (unifier) or **FAIL**.

VIII. Resources Required

S. No.	Resource	Suggested Broad Specification	Quantity
1	Computer System	Intel i5 or higher, 8 GB RAM, 256 GB SSD, Python 3.x installed	1
2	Prolog Programming Environment	SWI-Prolog, latest stable version	Installed
3	Prolog IDE / Editor	SWI-Prolog IDE or Visual Studio Code with Prolog extension	1

IX. Safety Precautions

a. ***Avoid*** sharing login credentials.
b. ***Ensure*** antivirus software and firewalls are updated.
c. ***Keep*** eatables away from computers.
d. ***Follow*** Institutional laboratory policies.
e. ***Avoid*** storing confidential data in lab computers.

X. Procedure *(Self-Instructional)*

1. ***Interpret*** the unification algorithm to understand variable matching and substitution in Prolog.
2. ***Specify*** the input expressions as valid Prolog terms with appropriate predicates and arguments.
3. ***Plan*** the unification process by identifying variables, constants, and compound terms.
4. ***Load*** the Prolog environment such as SWI-Prolog for coding and testing.
5. ***Define*** facts and sample terms required for the unification experiment.
6. ***Represent*** the expressions as Prolog terms and use the = operator for unification.
7. ***Execute*** queries to unify variables with constants, variables, and compound terms.
8. ***Handle*** incompatible terms and observe Prolog returning false.
9. ***Display*** the resulting variable bindings produced by successful unification.
10. ***Test*** the program with different combinations of constants, variables, and nested terms.
11. ***Analyze*** successful and failed unification cases based on Prolog's responses.
12. ***Document*** the input expressions, queries, substitutions, and results obtained.

XI. Actual Resources Used (*To be filled by the students*)

S. No	Name of Resource	Version/Configuration	Remarks
1.	Operating System	Windows 10/11	Used for program execution
2.	Programming Language	Python 3.x	Used to implement unification
3.	Software Development Kit	Python 3.x	Used for development and execution
4.	Libraries	No external libraries	Built-in Python features used

XII. Actual Procedure Followed (*To be filled by the students*)

1. Defined logical expressions, predicates, variables, and constants for the Food Knowledge Base.
2. Implemented the unification process to compare terms and generate suitable variable substitutions.
3. Tested the program using different queries involving constants, variables, and predicates.

4. Verified the obtained substitutions and identified successful and failed unification cases.

5. Applied unification concepts to the College Library scenario and analyzed the results.

Questions:

Scenario: College Library

Facts :

1. Ravi is a student.
2. Priya is a student.
3. Anu is a student.
4. Kiran is a student.
5. Ravi borrows Artificial Intelligence.
6. Ravi borrows Computer Networks.
7. Priya borrows Database.
8. Anu borrows Cyber Security.
9. Artificial Intelligence is a Computer Science book.
10. Computer Networks is a Computer Science book.

Rules:

1. Every student is a library member.
2. A student is a reader if the student borrows a book.
3. A student is a Computer Science reader if the student borrows a Computer Science book.
4. A student is an active reader if the student borrows two different books.

Unification Queries:

1. Find the student who borrows Artificial Intelligence.
2. Find the books borrowed by Ravi.
3. Find all student-book borrowing pairs.
4. Find the students who borrow Computer Science books.
5. Find all library members.
6. Find the active readers.

Output:

1. Query:

?- borrows(Student, artificial_intelligence).

Answer:

```
35 ?-
|    borrows(Student, artificial_intelligence).
Student = ravi.
```

2. Find the books borrowed by Ravi.

Query:
?- borrows(ravi, Book).

Answer:

```
36 ?-
|
|    borrows(ravi, Book).
Book = artificial_intelligence
```

3. Find all student-book borrowing pairs.

Query:
?- borrows(Student, Book).

Answers:

```
39 ?- borrows(Student, Book).
Student = ravi,
Book = artificial_intelligence ;
Student = ravi,
Book = computer_networks ;
Student = priya,
Book = database ;
Student = anu,
Book = cyber_security.
```

4.Find the students who borrow Computer Science books.

Query:
?- cs_reader(Student).

Answer:

```
51 ?- cs_reader(Student).
Student = ravi ;
```

5.Find all library members.

Query:
?- member(Student).

Answers:

```
64 ?- library_member(Student).
Student = ravi ;
Student = priya ;
Student = anu ;
Student = kiran.
```

6.Find the active readers.

Query:
?- active_reader(Student).

Answer:

```
65 ?- active_reader(Student).
Student = ravi ;
```

XIII. Observations *(To be filled by the students)*

S. No.	Line Number	Syntax Error	Semantic Error	Exception
1	25	Missing full stop . at the end of a Prolog fact	-	-
2	35	-	Undefined predicate cs_reader/1	Unknown procedure error

XIV. Results/Observations *(To be filled by the students)*

The unification process successfully matches variables with corresponding constants or terms when the logical expressions are compatible.

When two constants or predicates cannot be matched, unification fails and no valid substitution is produced.

XV. Interpretation of Results *(Students to state the meaning of the above-obtained results/observations)*

The unification process successfully matched logical expressions by finding suitable substitutions for variables. The results showed that variables can be unified with constants or other terms when the expressions are compatible.

The experiment also demonstrated unification through facts and rules. Queries were successfully used to identify persons, foods, books, and relationships from the given knowledge bases.

Thus, the obtained results confirm that unification is useful for logical inference, rule-based reasoning, knowledge representation, and Prolog-based AI systems.

XVI. Conclusions *(Students to draw conclusions (or take decisions) based on the interpretation of results)*

The unification algorithm was successfully implemented and tested using different logical expressions.

The experiment demonstrated variable substitution, matching of constants, multiple-variable unification, and rule-based inference.

Unification is an important technique in Artificial Intelligence, Knowledge Representation, Prolog, expert systems, and automated reasoning.

The experiment also showed that incompatible terms cannot be unified and result in failure.

XVII. Suggested Practicum Related Questions:

Note: *Below are a few questions related to industry-specific higher-order thinking skills (HOT) that teachers must use to ensure students achieve the predetermined course outcomes.*

1. What is unification in the context of logical inference and how does it support rule-based reasoning in AI?

Answer:
Unification is the process of making two logical expressions identical by finding suitable substitutions for variables. It supports rule-based reasoning by matching queries with facts and rules to derive conclusions.

2. How does the implemented unification algorithm handle variable substitutions and conflicts between terms?

Answer:
The algorithm substitutes a variable with the corresponding constant or term. If two constants are different or the terms are incompatible, the algorithm reports failure instead of creating an invalid substitution.

3. Explain the difference between syntactic and semantic unification with examples.

Answer:
Syntactic unification matches expressions based on their structure and symbols.

Example:

eats(Person, apple)
eats(john, apple)

Unification gives:

Person = john

Semantic reasoning considers the meaning or interpretation of knowledge rather than only the structure of expressions.

4. What are the conditions for two logical expressions to be unifiable?

Answer:
Two expressions are unifiable when they have compatible structures, the same predicate or function name where required, the same number of arguments, and their variables can be assigned values consistently. Conflicting constants make unification fail.

5. Describe how the algorithm handles expressions with nested structures or multiple variables.

Answer:
The algorithm compares compound expressions recursively. It checks their function names and then compares their corresponding arguments. Multiple variables are assigned substitutions consistently throughout the expressions.

6. What challenges can arise when unifying predicates with different arities or function symbols?

Answer:
Predicates with different numbers of arguments cannot normally be unified. Similarly, different function symbols are incompatible. For example:

eats(john, rice)
eats(john, rice, food)

cannot be unified because their arities are different.

7. How can the unification process be integrated into expert systems or Prolog-based environments?

Answer:
Unification can match user queries with stored facts and rules. In Prolog, it is used to find variable bindings while executing logical queries, allowing the system to derive conclusions automatically.

8. Evaluate the time complexity of your unification algorithm and suggest optimizations.

Answer:
For simple terms, unification generally requires time proportional to the number of terms being compared, approximately **O(n)**. Efficiency can be improved using efficient data structures, substitution tracking, and avoiding repeated comparisons.

9. How does unification contribute to backward and forward chaining in inference engines?

Answer:
In **backward chaining**, unification matches a goal with the conclusion of a rule and determines the required substitutions. In **forward chaining**, it matches known facts with rule conditions to derive new facts.

10. Can unification be extended to support fuzzy or probabilistic reasoning in AI systems?

Answer:
Yes. Unification can be combined with fuzzy or probabilistic systems by associating degrees of truth or probabilities with facts and rules. This allows reasoning when information is uncertain rather than completely true or false.

XVIII References / Suggestions for further reading

Online Tutorials & Articles

a. GeeksforGeeks – Unification in AI- https://www.geeksforgeeks.org/unification-in-artificial-intelligence/
b. Javatpoint – Unification in AI- https://www.javatpoint.com/unification-in-ai

Interactive Platforms for Practicing Sorting Problems

a. HackerRank – Artificial Intelligence (AI) Challenges- https://www.hackerrank.com/domains/tutorials/10-days-of-artificial-intelligence
b. Codecademy – Learn Prolog- https://www.codecademy.com/learn/learn-prolog

Video Lectures & Courses

a. Coursera – Logic for Artificial Intelligence by Stanford University- https://www.coursera.org/learn/logic-introduction
b. Learn Prolog Programming (University of York) https://youtu.be/nqvOcqLgQnM?si=S1EKkReYaXgnKnOn

Suggested Assessment Scheme

The performance indicators given serve as a guideline for assessing the *'process-related skills/LOs'* (marks to be awarded in real-time in the laboratory by the faculty member, as these cannot be measured after the practicum is over) and *'product-related skills/LOs'*

Note: The weightage for the ***process-related skills*** and ***product-related skills*** will change depending on the PrO, COs, and the competency related to the concerned course.

Performance Indicators	Max. Marks	Marks Obtained
Process related skills: 18 Marks - 60%		
Understanding of Intelligent agents	3	
Application of suitable AI technique/ algorithm	6	
Followed Coding Standards and Best Practices	6	
Implementation of AI agent correctly	3	
Product related skills: 12 Marks - 40%		
Accuracy of Output/Agent behaviour/ model predictions	3	
Evaluation of performance	3	
Error Handling & Debugging	1.5	
Interpretation of Results and Conclusions	1.5	
Practical related questions	1.5	
Submitting the Report on time	1.5	
Total	**30**	

To be filled by Student:

S.No.	Name of the Student	Register No.
1	**Aashin Surjith Thampi C**	**URK25CS1227**

To be filled by the Faculty Member:

Marks Obtained			Name and Designation of the Faculty Member	Date of Evaluation
Process-Related Skills (18)	**Product-Related Skills (12)**	**Total (30)**		

****

ARISE AND SHINE
Karunya